

1 Pandas Exercises

```
[1]: import numpy as np
import pandas as pd

data: dict[str, list[str | float]] = {
    'birds': ['Cranes', 'Cranes', 'plovers', 'spoonbills', 'spoonbills', 'Cranes', 'plovers', 'Cranes', 'spoonbills', 'spoonbills'],
    'age': [3.5, 4, 1.5, np.nan, 6, 3, 5.5, np.nan, 8, 4],
    'visits': [2, 4, 3, 4, 3, 4, 2, 2, 3, 2],
    'priority': ['yes', 'yes', 'no', 'yes', 'no', 'no', 'no', 'yes', 'no', 'no']
}
labels: list[str] = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
```

1.1 Create a DataFrame birds from this dictionary data which has the index labels.

```
[2]: df: pd.DataFrame = pd.DataFrame(data, labels)
df
```

```
[2]:
```

	birds	age	visits	priority
a	Cranes	3.5	2	yes
b	Cranes	4.0	4	yes
c	plovers	1.5	3	no
d	spoonbills	NaN	4	yes
e	spoonbills	6.0	3	no
f	Cranes	3.0	4	no
g	plovers	5.5	2	no
h	Cranes	NaN	2	yes
i	spoonbills	8.0	3	no
j	spoonbills	4.0	2	no

1.2 Display a summary of the basic information about birds DataFrame and its data.

```
[3]: df.info()

<class 'pandas.DataFrame'>
Index: 10 entries, a to j
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype  
---  -
 0   birds       10 non-null    str     
 1   age         8 non-null     float64
 2   visits      10 non-null    int64   
 3   priority    10 non-null    str     
dtypes: float64(1), int64(1), str(2)
memory usage: 400.0+ bytes
```

1.3 Print the first 2 rows of the birds dataframe.

```
[4]: df.head(2)
```

```
[4]:      birds  age  visits priority
a  Cranes  3.5      2      yes
b  Cranes  4.0      4      yes
```

1.4 Print all the rows with only 'birds' and 'age' columns from the dataframe.

```
[5]: df[["birds", "age"]]
```

```
[5]:      birds  age
a    Cranes  3.5
b    Cranes  4.0
c  plovers  1.5
d spoonbills NaN
e spoonbills 6.0
f    Cranes  3.0
g  plovers  5.5
h    Cranes  NaN
i spoonbills 8.0
j spoonbills 4.0
```

1.5 Select [2, 3, 7] rows and in columns ['birds', 'age', 'visits'].

```
[6]: df.iloc[[2, 3, 7]][['birds', 'age', 'visits']]
```

```
[6]:      birds  age  visits
c  plovers  1.5      3
d spoonbills NaN      4
h    Cranes  NaN      2
```

1.6 Select the rows where the number of visits is less than 4.

```
[7]: df[df["visits"] < 4]
```

```
[7]:      birds  age  visits priority
a    Cranes  3.5      2      yes
c  plovers  1.5      3      no
e spoonbills 6.0      3      no
g  plovers  5.5      2      no
h    Cranes  NaN      2      yes
i spoonbills 8.0      3      no
j spoonbills 4.0      2      no
```

1.7 Select the rows with columns ['birds', 'visits'] where the age is missing i.e NaN.

```
[8]: df[df["age"].isna()][["birds", "visits"]]
```

```
[8]:      birds  visits
d  spoonbills      4
h      Cranes      2
```

1.8 Select the rows where the birds is a Cranes and the age is less than 4.

```
[9]: df[(df["birds"] == "Cranes") & (df["age"] < 4)]
```

```
[9]:      birds  age  visits priority
a  Cranes  3.5      2      yes
f  Cranes  3.0      4      no
```

1.9 Select the rows the age is between 2 and 4(inclusive).

```
[10]: df[df["age"].between(2, 4, "both")]
```

```
[10]:      birds  age  visits priority
a   Cranes  3.5      2      yes
b   Cranes  4.0      4      yes
f   Cranes  3.0      4      no
j  spoonbills  4.0      2      no
```

1.10 Find the total number of visits of the bird Cranes.

```
[11]: print(df[df["birds"] == "Cranes"]["visits"].sum())
```

12

1.11 Calculate the mean age for each different birds in dataframe.

```
[12]: df.groupby("birds")["age"].mean()
```

```
[12]: birds
Cranes      3.5
plovers     3.5
spoonbills  6.0
Name: age, dtype: float64
```

1.12 Append a new row 'k' to dataframe with your choice of values for each column. Then delete that row to return the original DataFrame.

```
[13]: df.loc["k"] = ["parrot", 3, 10, "yes"]
df
```

```
[13]:
```

	birds	age	visits	priority
a	Cranes	3.5	2	yes
b	Cranes	4.0	4	yes
c	plovers	1.5	3	no
d	spoonbills	NaN	4	yes
e	spoonbills	6.0	3	no
f	Cranes	3.0	4	no
g	plovers	5.5	2	no
h	Cranes	NaN	2	yes
i	spoonbills	8.0	3	no
j	spoonbills	4.0	2	no
k	parrot	3.0	10	yes

```
[14]: df.drop("k", inplace=True)
df
```

```
[14]:
```

	birds	age	visits	priority
a	Cranes	3.5	2	yes
b	Cranes	4.0	4	yes
c	plovers	1.5	3	no
d	spoonbills	NaN	4	yes
e	spoonbills	6.0	3	no
f	Cranes	3.0	4	no
g	plovers	5.5	2	no
h	Cranes	NaN	2	yes
i	spoonbills	8.0	3	no
j	spoonbills	4.0	2	no

1.13 Find the number of each type of birds in dataframe (Counts).

```
[15]: df.groupby("birds")["age"].count().rename("count")
```

```
[15]: birds
Cranes      3
plovers     2
spoonbills  3
Name: count, dtype: int64
```

1.14 Sort dataframe (birds) first by the values in the 'age' in descending order, then by the value in the 'visits' column in ascending order.

```
[16]: df.sort_values("age", ascending=False).sort_values("visits")
```

```
[16]:      birds  age  visits  priority
g  plovers  5.5      2      no
j  spoonbills  4.0      2      no
a   Cranes  3.5      2     yes
h   Cranes  NaN      2     yes
c  plovers  1.5      3      no
e  spoonbills  6.0      3      no
i  spoonbills  8.0      3      no
b   Cranes  4.0      4     yes
f   Cranes  3.0      4      no
d  spoonbills  NaN      4     yes
```

1.15 Replace the priority column values with 'yes' should be 1 and 'no' should be 0.

```
[17]: df["priority"] = df["priority"].map({"no": 0, "yes": 1})
df
```

```
[17]:      birds  age  visits  priority
a   Cranes  3.5      2         1
b   Cranes  4.0      4         1
c  plovers  1.5      3         0
d  spoonbills  NaN      4         1
e  spoonbills  6.0      3         0
f   Cranes  3.0      4         0
g  plovers  5.5      2         0
h   Cranes  NaN      2         1
i  spoonbills  8.0      3         0
j  spoonbills  4.0      2         0
```

1.16 In the 'birds' column, change the 'Cranes' entries to 'trumpeters'.

```
[18]: df["birds"] = df["birds"].replace("Cranes", "trumpeters")
df
```

```
[18]:      birds  age  visits  priority
a  trumpeters  3.5      2         1
b  trumpeters  4.0      4         1
c  plovers  1.5      3         0
d  spoonbills  NaN      4         1
e  spoonbills  6.0      3         0
f  trumpeters  3.0      4         0
g  plovers  5.5      2         0
h  trumpeters  NaN      2         1
i  spoonbills  8.0      3         0
j  spoonbills  4.0      2         0
```